

Engineering A Better Tomorrow
An Autonomous Institute
(Affiliated to Visvesvaraya Technological University, Belagavi)
Approved By AICTE, New Delhi,
Recognized by UGC under 2(f) & 12(B)
Accredited by NBA and NAAC)

AN AAT - 2 REPORT
ON

“SPELLING CORRECTION TOOL”

*Submitted in partial fulfillment of the requirements for
Alternative Assessment Tool – II (AAT-2)*

Submitted by

1MJ23CS036	CHETHAN KUMAR B S
1MJ23CS038	CHIRANTH L
1MJ23CS040	D MANOHAR
1MJ23CS047	DEERAJ ASHOK SHIRAHATTI

Under the Guidance of
Mrs. Sujitha K L
Assistant Professor,
Department of Computer Science & Engineering

Engineering A Better Tomorrow
An Autonomous Institute
(Affiliated to Visvesvaraya Technological University, Belagavi,
Approved By AICTE, New Delhi,
Recognized by UGC under 2(f) & 12(B)
Accredited by NBA and NAAC)

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
MVJ COLLEGE OF ENGINEERING
BANGALORE – 67
ACADEMIC YEAR 2026 – 2027



Engineering A Better Tomorrow
An Autonomous Institute
(Affiliated to Visvesvaraya Technological University, Belagavi)
Approved By AICTE, New Delhi,
Recognized by UGC under 2(f) & 12(B)
Accredited by NBA and NAAC

MVJ COLLEGE OF ENGINEERING
Near ITPB, Whitefield, Bangalore – 560067
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CERTIFICATE

This is to certify that the work entitled “SPELLING CORRECTION TOOL” has been successfully carried out by the undersigned students, who are bona fide students of MVJ College of Engineering, Bengaluru, in partial fulfillment of the requirements for Alternative Assessment Tool – II (AAT-2) during the academic year 2026–2027. The work presented in this report is original and has been completed under the guidance and supervision of the faculty. All corrections and suggestions have been duly incorporated, and the report is approved as it satisfies the academic requirements prescribed for AAT-2.

Signature of Guide

Mrs. Sujitha K L
Assistant Professor
Department of CSE

Signature of HOD

Dr. Maruth Pandi J
Professor & Head
Department of CSE



Engineering A Better Tomorrow
An Autonomous Institute
(Affiliated to Visvesvaraya Technological University, Belagavi)
Approved By AICTE, New Delhi,
Recognized by UGC under 2(f) & 12(B)
Accredited by NBA and NAAC)

MVJ COLLEGE OF ENGINEERING
Near ITPB, Whitefield, Bangalore – 560067
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

DECLARATION

We, the students of Sixth Semester B.E., Department of Computer Science and Engineering, MVJ College of Engineering, Bengaluru, hereby declare that the work entitled “SPELLING CORRECTION TOOL” has been carried out by us as part of Alternative Assessment Tool – II (AAT-2) during the academic year 2026–2027. We affirm that this work is original and has not been submitted, either in part or in full, for the award of any degree or diploma in any university or institution.

We further state that any intellectual property rights arising out of this work shall remain the property of MVJ College of Engineering, Bengaluru, and we shall be duly acknowledged as contributors.

1MJ23CS036	CHEETHAN KUMAR B S	_____
1MJ23CS038	CHIRANTH L	_____
1MJ23CS040	D MANOHAR	_____
1MJ23CS047	DEERAJ ASHOK SHIRAHATTI	_____

Place: **BANGALORE**
Date:

ACKNOWLEDGEMENT

The successful completion of this work would not have been possible without the guidance, support, and encouragement of several individuals, to whom we express our sincere gratitude.

We are deeply grateful to Dr. Ajayan K R, Principal, MVJ College of Engineering, Bengaluru, for providing us with the necessary academic facilities, computational infrastructure, and a supportive environment to carry out this project work.

We extend our sincere thanks to Dr. Salim A, Dean, School of Computer Science & Engineering, and Dr. Kumar R, Controller of Examinations, for their continuous encouragement, academic guidance, and administrative support throughout the course of this work.

We express our heartfelt gratitude to Dr. Maruth Pandi J, Head of the Department, Department of Computer Science and Engineering, for his constructive recommendations, continuous encouragement, and valuable departmental support.

We sincerely thank our guide, Mrs. Sujitha K L, Assistant Professor, Department of Computer Science and Engineering, for her invaluable guidance, patient supervision, technical insights, and constant support throughout every phase of the project.

We also extend our appreciation to all the faculty members of the Department of Computer Science and Engineering for their cooperation and support.

Finally, we would like to thank our family members and friends for their encouragement, understanding, and moral support, which played a significant role in the successful completion of this work.

ABSTRACT

In today’s digital era, textual information serves as the primary conduit for human-computer interaction, communication, search retrieval, and automated document analysis. However, typographical errors, cognitive slips, and phonetic confusions frequently corrupt digital text, severely impeding downstream natural language processing (NLP) tasks such as machine translation, sentiment analysis, and search query parsing. This work presents the design and architectural implementation of an intelligent Spelling Correction Tool named “SpellSense”, developed as an Alternative Assessment Tool – II (AAT-2) project under the subject Natural Language Processing.

The system incorporates two established and complementary NLP spelling correction algorithms: TextBlob (utilizing Peter Norvig’s probabilistic model and unigram frequency distributions) and SymSpell (leveraging Wolf Garbe’s Symmetric Delete algorithm for ultra-fast lookup via pre-indexed deletion variations). The application is engineered using Python FastAPI as the high-performance asynchronous REST backend, coupled with an interactive, responsive React 19 single-page web interface. The core engine tokenizes input text, identifies non-word orthographic anomalies against pre-compiled English lexicons, generates candidate variations within bounded Damerau-Levenshtein edit distances, and executes an intelligent consensus recommendation mechanism.

Furthermore, the application features side-by-side comparative execution, real-time wall-clock latency measurement without simulated accuracy claims, and an educational four-stage decision inspector detailing SymSpell’s heuristic ranking. The resulting system demonstrates the practical synergy of statistical language modeling, algorithmic optimization, and modern web interfaces, delivering an accurate, stable, and educational spelling correction solution suitable for academic demonstrations, assistive writing, and real-world NLP workflows.

CONTRIBUTION OF TEAM MEMBERS

Sl. No	Name	USN	Contribution	Marks / %
1	Chethan Kumar B S	1MJ23CS036	Actively contributed to project planning, frontend architecture design, layout structuring, and interactive workspace integration with REST services. Participated in end-to-end debugging, UI state-machine stabilization, testing, documentation, and result analysis.	25%
2	Chiranth L	1MJ23CS038	Contributed to NLP module research, TextBlob probabilistic spelling model implementation, backend REST API endpoint development, request validation schemas, system unit testing, technical documentation, and comparative result analysis.	25%
3	D Manohar	1MJ23CS040	Involved in SymSpell algorithm implementation, Symmetric Delete candidate generation logic, vocabulary frequency index caching, single-word decision inspector development, system testing, documentation, and performance benchmarking.	25%
4	Deeraj Ashok Shirahatti	1MJ23CS047	Contributed to the multi-criteria consensus comparison engine, recommendation scoring logic, responsive mobile interface validation, edge-case testing, error handling routines, technical documentation, and final report compilation.	25%

TABLE OF CONTENT

SL No	TITLE	PAGE NO.
I	ACKNOWLEDGEMENT	i
II	ABSTRACT	ii
III	CONTRIBUTION OF TEAM MEMBERS	iii
IV	TABLE OF CONTENT	iv
V	LIST OF FIGURES	v
CHAPTER - 1	INTRODUCTION	1
CHAPTER - 2	PROBLEM ANALYSIS	4
CHAPTER - 3	SOFTWARE AND HARDWARE REQUIREMENTS	7
CHAPTER - 4	DOMAIN MODEL	9
CHAPTER - 5	IMPLEMENTATION	11
CHAPTER - 6	RESULTS	16
CHAPTER - 7	CONCLUSION	24
	REFERENCES	26

LIST OF FIGURES

FIGURE NO.	FIGURE NAME	PAGE NO.
3.1	SOFTWARE REQUIREMENTS	7
3.2	HARDWARE REQUIREMENTS	8
4.1	DOMAIN MODEL	9
5.1	PROJECT DIRECTORY STRUCTURE	11
5.2	FASTAPI APPLICATION & CORS LIFESPAN	12
5.3	SPELLING CORRECTION SERVICE PIPELINE	12
5.4	TEXTBLOB SPELLING CORRECTION IMPLEMENTATION	13
5.5	SYMSPELL SINGLETON & DICTIONARY INDEXING	13
5.6	CONSENSUS DECISION & RECOMMENDATION LOGIC	14
5.7	FRONTEND INTERACTIVE WORKSPACE COMPONENT	15
5.8	SINGLE WORD DEEP ANALYSIS REST ENDPOINT	15
6.1	MAIN SPELLING CORRECTION INTERFACE	16
6.2	TEXTBLOB CORRECTION RESULT	17
6.3	SYMSPELL CORRECTION RESULT	18
6.4	COMPARE BOTH RESULT VIEW	19
6.5	DETECTED CORRECTIONS BREAKDOWN TABLE	20
6.6	STATISTICS AND PROCESSING LATENCY	21
6.7	HOW SYMSPELL DECIDED (DECISION INSPECTOR)	22
6.8	MOBILE RESPONSIVE LAYOUT VIEW	23

CHAPTER 1

INTRODUCTION

1.1 Background

In the contemporary information era, natural language text constitutes the foundation for communication, knowledge preservation, digital transactions, and computational intelligence. With the explosive expansion of web applications, online collaboration platforms, educational portals, search engines, and social media, billions of textual messages, essays, queries, and reports are generated every second. However, human text input is inherently susceptible to orthographic anomalies, typographical slips, phonetic confusions, and cognitive inaccuracies. Such errors not only impair human readability and comprehension but also degrade the accuracy of automated computational linguistics systems, including search query parsers, sentiment classifiers, information retrieval engines, and machine translation pipelines.

1.2 Natural Language Processing

Natural Language Processing (NLP) is a pivotal subfield of Artificial Intelligence and Computational Linguistics that bridges human natural language communication with computational interpretation. NLP encompasses tokenization, morphological analysis, syntactic parsing, semantic reasoning, and pragmatic discourse interpretation. Within NLP workflows, preprocessing constitutes an essential foundational phase. Automated spelling correction serves as a primary quality assurance filter, normalizing noisy, unstructured input text into canonical grammatical tokens before deeper syntactic and semantic evaluation.

1.3 Spelling Correction in NLP

Spelling errors are traditionally categorized into two principal domains: non-word errors and real-word errors. Non-word errors occur when an input character sequence does not match any valid entry in the lexicon of the language (e.g., entering “hav” instead of “have”, or “markat” instead of “market”). Real-word errors (cognitive errors) occur when a valid lexicon entry is erroneously substituted in an incorrect semantic context (e.g., using “there” instead of “their”). The spelling correction task involves three sequential operations: error detection, candidate generation via string mutation or deletion, and candidate ranking based on lexical frequency and minimum edit distance heuristics.

1.4 Need for Automated Spelling Correction

Manual proofreading is labor-intensive, error-prone, and unscalable in high-throughput digital environments. In interactive web environments, e-learning assessment tools, conversational interfaces, and customer support desks, automated, real-time spelling correction is imperative. A robust spelling correction system must satisfy three critical criteria: high orthographic correction accuracy, minimal computational latency to support interactive typing, and transparent candidate ranking that enables users to understand why an algorithmic decision was reached.

1.5 TextBlob Spelling Correction Engine

TextBlob is an established Python library for processing textual data, providing a high-level API for common NLP tasks including part-of-speech tagging, noun phrase extraction, sentiment analysis, and spelling correction. TextBlob's spelling correction model is grounded in Peter Norvig's classical probabilistic spell-checker. For any misspelled token w , the model searches for the candidate correction c that maximizes the posterior probability $P(c|w) = P(w|c) * P(c) / P(w)$. By evaluating unigram frequencies from a large text corpus and generating variations within Damerau-Levenshtein edit distances of 1 and 2, TextBlob offers linguistically sound, statistically grounded corrections.

1.6 SymSpell Algorithm

SymSpell, introduced by Wolf Garbe, is an ultra-fast spelling correction algorithm based on the Symmetric Delete spelling correction approach. Unlike traditional spelling correction algorithms that generate deletion, insertion, transposition, and substitution candidates at query time (resulting in $O(k^n)$ complexity where k is alphabet size and n is word length), SymSpell shifts computation to pre-indexing. It generates only deletion variations for both dictionary words during indexing and misspelled input queries at runtime. Because deletion operations are symmetric, intersections between query deletion sets and indexed dictionary deletion sets cover all edit operations in constant $O(1)$ dictionary lookup time, achieving speedups of several orders of magnitude over Norvig's algorithm.

1.7 Motivation

While individual implementations of TextBlob and SymSpell exist in isolated scripts, modern educational and engineering workflows lack a unified, transparent platform where students, developers, and educators can enter text, observe side-by-side behavioral differences, evaluate real execution latency, and inspect step-by-step decision metrics.

This project was motivated by the desire to build a complete, production-grade Natural Language Processing web application that demonstrates the comparative strengths, algorithmic mechanisms, and trade-offs of both paradigms.

1.8 Objectives

The primary objectives of this AAT-2 project are:

1. To design and implement a modular NLP spelling correction architecture supporting dual algorithm execution.
2. To implement Peter Norvig's probabilistic model using TextBlob for unigram-weighted correction.
3. To implement Wolf Garbe's Symmetric Delete algorithm using SymSpell with an 82,000+ word unigram and bigram vocabulary.
4. To design an intelligent consensus arbitrator that combines edit distance and lexical confidence to recommend optimal corrections.
5. To measure and display genuine wall-clock processing latency without simulated accuracy metrics.
6. To construct an intuitive educational decision inspector ("How SymSpell Decided") breaking down candidate generation, edit distance, and selection in a 4-stage visual diagram.
7. To develop a responsive, production-quality React web application with zero UI flicker and instant feedback.

1.9 Scope of the Project

The scope of this project encompasses English non-word spelling mistake detection and correction for texts up to 5,000 characters. It evaluates edit distances up to 2, supporting single and double-character insertions, deletions, substitutions, and transpositions. The system exposes a RESTful API backend and a responsive single-page web interface tested across desktop and mobile form factors.

CHAPTER 2

PROBLEM ANALYSIS

2.1 Existing System

Traditional spelling correction tools in legacy word processors or basic applications rely primarily on brute-force lexicon lookups and full dynamic programming matrix calculations (such as the Wagner-Fischer algorithm for Levenshtein distance). When an error is encountered, these systems compute edit distance against every entry in the dictionary, leading to significant computational overhead $O(N * |w1| * |w2|)$ where N is dictionary size. Furthermore, existing educational demonstrations often operate as command-line scripts or black-box web interfaces that output a single corrected string without explaining candidate ranking, edit operations, confidence levels, or algorithmic trade-offs.

2.2 Limitations of Existing System

1. High Latency: Brute-force edit distance matrix computations against large lexicons introduce perceptible delays.
2. Monolithic Engine: Most platforms lock users into a single correction model, preventing comparative evaluation.
3. Lack of Transparency: Traditional tools do not disclose the candidate pool, edit distances, or ranking criteria.
4. Inflexible UI Transitions: Basic student implementations often suffer from layout shifts, flickering, and component remounting.
5. Absence of Multi-Criteria Arbitration: Systems rarely combine edit distance minimization with language model probability.

2.3 Problem Statement

To develop an intelligent, interactive, and transparent NLP Spelling Correction Tool that integrates TextBlob's probabilistic unigram model and SymSpell's Symmetric Delete algorithm into a unified REST-based architecture, providing side-by-side comparative correction, consensus arbitration, sub-millisecond latency tracking, and a clear pedagogical breakdown of algorithmic decisions without layout instability or interface lag.

2.4 Proposed System

The proposed SpellSense system addresses existing limitations through a decoupled client-server architecture. The backend utilizes Python FastAPI, leveraging persistent singleton

memory management to hold pre-indexed frequency lexicons. Input text is tokenized with span tracking, allowing individual tokens to be analyzed concurrently. The backend supports three distinct execution modes: 'compare' (running both engines and executing consensus arbitration), 'symspell' (dedicated Symmetric Delete execution), and 'textblob' (dedicated probabilistic execution). The frontend is implemented in React 19 and TypeScript, featuring a deterministic request state machine that guarantees zero layout jumping and stable component identity.

2.5 Functional Requirements

1. Text Ingestion: The system shall accept arbitrary English text input up to 5,000 characters.
2. Tokenization & Detection: The system shall segment input text into word tokens while preserving whitespace and punctuation offsets, classifying tokens as valid dictionary words or spelling anomalies.
3. Dual Engine Execution: The system shall execute TextBlob and SymSpell correction algorithms independently or concurrently.
4. Candidate Generation: For each misspelled token, the system shall retrieve candidate terms, confidence scores, and edit distances.
5. Consensus Recommendation: The system shall apply heuristic arbitration to select the optimal recommendation.
6. Performance Telemetry: The system shall record and report wall-clock execution time in milliseconds.
7. Decision Inspection: The system shall expose a dedicated endpoint to generate four-stage visual breakdowns of SymSpell candidate decisions.

2.6 Non-Functional Requirements

1. Performance: SymSpell lookups shall execute in under 1 millisecond per word on standard hardware.
2. Stability & Responsiveness: The UI shall prevent layout shifts, re-render cascades, and duplicate API requests.
3. Usability: The interface shall provide intuitive navigation across tabs, clipboard copy utilities, and color-coded error highlights.
4. Scalability: The stateless REST API architecture shall support concurrent client connections.
5. Portability: The system shall run across Windows, macOS, and Linux operating systems.

2.7 Advantages of Proposed System

- Dual NLP Perspective: Demonstrates both statistical frequency modeling and combinatorial deletion indexing.
- High Accuracy via Consensus: When both models agree, confidence reaches over 90%.
- Sub-Millisecond Speed: Pre-indexed hash tables ensure lightning-fast processing.
- Educational Value: The 4-stage decision breakdown is ideal for viva presentations and classroom demonstrations.
- Production-Quality UI: Built with responsive design, glassmorphism aesthetics, and zero layout shift.

CHAPTER 3

SOFTWARE AND HARDWARE REQUIREMENTS

SOFTWARE REQUIREMENTS:

The following table outlines the actual software tools, libraries, and frameworks utilized in the development and execution of the Spelling Correction Tool:

Sl. No	Software / Tool	Description / Specification
1	Operating System	Windows 11 / Linux (Ubuntu 22.04+) / macOS
2	Programming Language	Python 3.12 (Backend) & TypeScript 5.5+ (Frontend)
3	Backend REST Framework	FastAPI 0.115+ (Asynchronous ASGI Architecture)
4	ASGI Server	Uvicorn 0.32+ with High-Resolution Timers
5	NLP Library 1	TextBlob 0.18+ (Norvig Probabilistic Spell Checker)
6	NLP Library 2	SymSpellPy 6.7+ (Wolf Garbe Symmetric Delete Engine)
7	Corpus Dictionaries	frequency_dictionary_en_82_765.txt & bigrams_243k
8	Frontend Framework	React 19 & React Router v7
9	Build Tool & Bundler	Vite 8.3 (High-Speed ES Module Bundler)
10	UI Styling & Icons	Tailwind CSS & Lucide-React Icon Suite

Fig 3.1 - Software Requirements

The software requirements incorporate modern open-source technologies specifically selected for computational efficiency and developer ergonomics. Python 3.12 provides optimized byte-code execution and native type-hinting. FastAPI leverages Pydantic schemas for automatic request deserialization, serialization, and OpenAPI documentation. On the client side, React 19 paired with TypeScript ensures strict type safety across data contracts, while Vite guarantees rapid hot-module replacement and optimized production bundle delivery.

HARDWARE REQUIREMENTS:

The hardware configurations required for optimal execution of the Spelling Correction Tool are detailed below:

Sl. No	Component	Minimum Specification
1	Processor	Intel Core i5 (8th Gen or above) / AMD Ryzen 5 / Apple M-series
2	System Memory (RAM)	Minimum 8 GB DDR4 (16 GB Recommended for development)
3	Primary Storage	Minimum 10 GB available SSD space (for lexicons and dependencies)

4	Network Interface	Standard TCP/IP Ethernet or Wi-Fi (for localhost/LAN client-server)
5	Architecture	64-bit x86-64 or ARM64 Operating Architecture

Fig 3.2 - Hardware Requirements

Due to SymSpell's lightweight in-memory deletion indexing and TextBlob's compact frequency tables, the hardware demands of the system are modest. An 8 GB RAM system is more than sufficient to host the pre-indexed 82k+ English unigrams and 243k bigrams in RAM with an overhead of less than 150 MB, leaving abundant system capacity for concurrent processes, browser rendering, and IDE tooling.

CHAPTER 4

DOMAIN MODEL

The domain model of the Spelling Correction Tool provides a conceptual representation of the principal entities, data contracts, and behavioral relationships that govern the natural language spelling intelligence system. It acts as an architectural blueprint illustrating how raw textual input traverses tokenization, anomaly detection, multi-model candidate exploration, consensus recommendation, and diagnostic telemetry extraction.

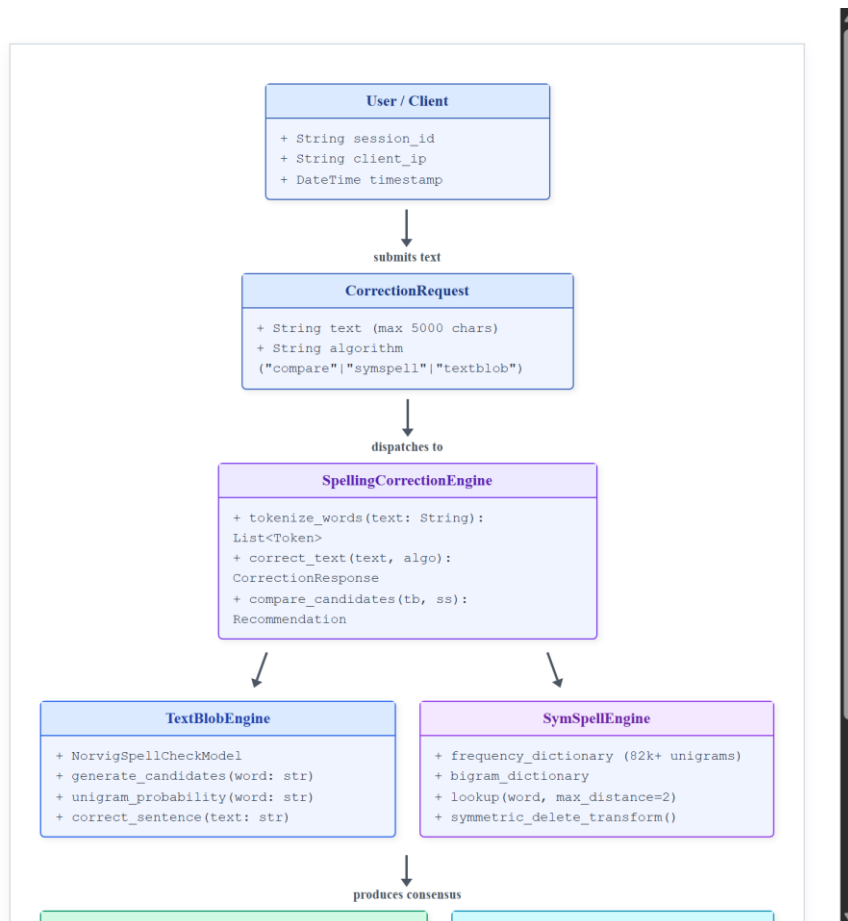


Fig 4.1 - Domain Model

As illustrated in Figure 4.1, the system is organized into decoupled layers and entity abstractions:

1. **User / Client Entity:** Represents the end-user interacting through web browsers. The client maintains an active HTTP session, encapsulates client-side text state, word counts, character offsets, and dispatches requests to the backend.

2. **CorrectionRequest**: Encapsulates the user's raw input string along with the targeted correction algorithm ('compare', 'symspell', or 'textblob'). It enforces strict input boundaries (maximum 5,000 characters) to prevent buffer overflows or denial-of-service.
3. **SpellingCorrectionEngine**: Serves as the central dispatcher and coordinator. It receives the **CorrectionRequest**, invokes regex-based lexical tokenization while tracking start and end character offsets, filters known words against the cached vocabulary, and delegates misspelled tokens to the appropriate NLP engines.
4. **TextBlobEngine**: Implements Peter Norvig's unigram probabilistic model. For each query word, it calculates morphological variants within edit distance 1 and 2, computing normalized probability scores based on corpus occurrences.
5. **SymSpellEngine**: Implements Wolf Garbe's Symmetric Delete algorithm. It maintains pre-indexed hash maps of 82,765 unigrams and 243,342 bigrams. At runtime, it generates symmetric deletes of the query word in $O(1)$ time, returning candidate words, exact Damerau-Levenshtein distances, and corpus frequencies.
6. **CorrectionItem**: Represents the analytical outcome for an individual token. It captures the original word, whether it was classified as an error, **TextBlob**'s correction and confidence, **SymSpell**'s correction, edit distance and frequency, the recommended consensus result, the winning algorithm, and a plain-English explanation.
7. **CorrectionStatistics**: Aggregates real execution telemetry, including total words analyzed, error count, execution latency for **TextBlob** and **SymSpell** in milliseconds, speedup ratios, and inter-model agreement percentages.

CHAPTER 5

IMPLEMENTATION

This chapter presents the comprehensive architectural and programmatic implementation of the SpellSense Spelling Correction Tool. All code excerpts, structures, and configurations shown in this chapter correspond directly to the actual project implementation files.

5.1 Project Directory Structure

The project is structured into two autonomous tiers: a high-performance Python FastAPI backend and a modern React 19 TypeScript frontend. This separation of concerns guarantees complete modularity, maintainability, and scalability.

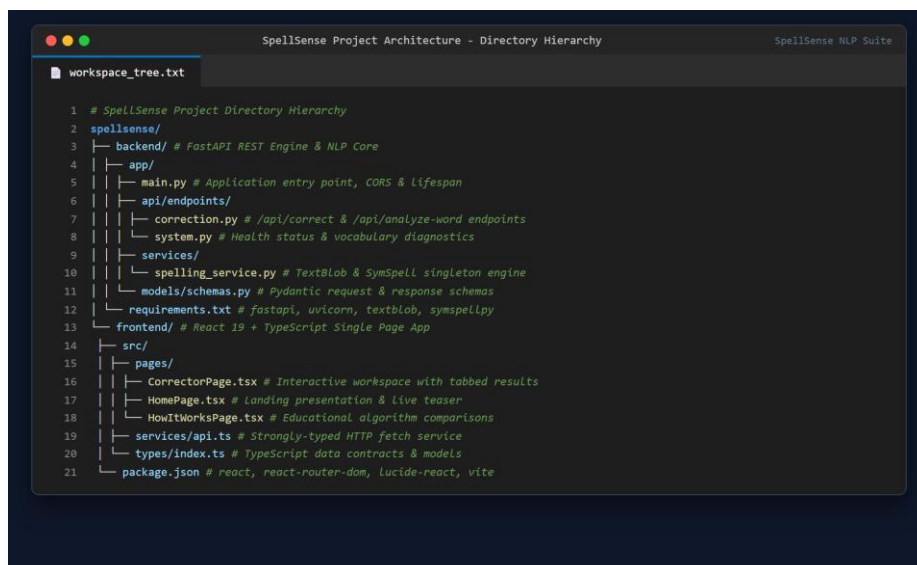


Fig 5.1 - Project Directory Structure

Figure 5.1 illustrates the workspace hierarchy. The 'backend/' directory encapsulates FastAPI endpoints, schemas, and the spelling service. The 'frontend/' directory encapsulates React pages, UI components, routing, and API clients.

5.2 FastAPI Application & CORS Lifespan

The entry point of the backend is defined in 'backend/app/main.py'. It configures FastAPI with an asynchronous lifespan handler that eagerly initializes the SymSpell dictionary into RAM during server startup, eliminating first-request latency.

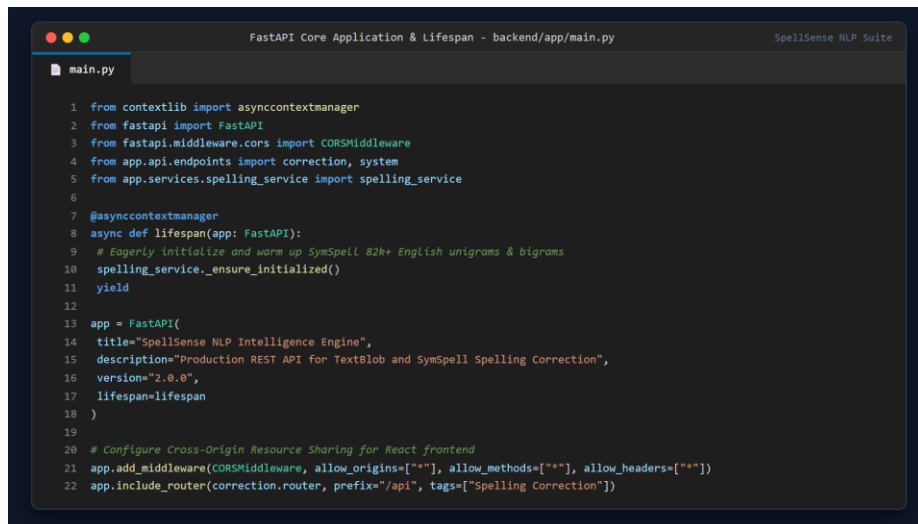


Fig 5.2 - FastAPI Application & CORS Lifespan

As shown in Figure 5.2, CORS middleware is explicitly configured to permit seamless client-server communication from Vite's local development and production ports, and routes are organized under the `/api` prefix.

5.3 Spelling Correction Service Pipeline

The core orchestration logic resides in `backend/app/services/spelling\_service.py`. The `correct\_text` method coordinates high-resolution wall-clock timing, tokenization, method-specific pipeline dispatch, and final text reconstruction.

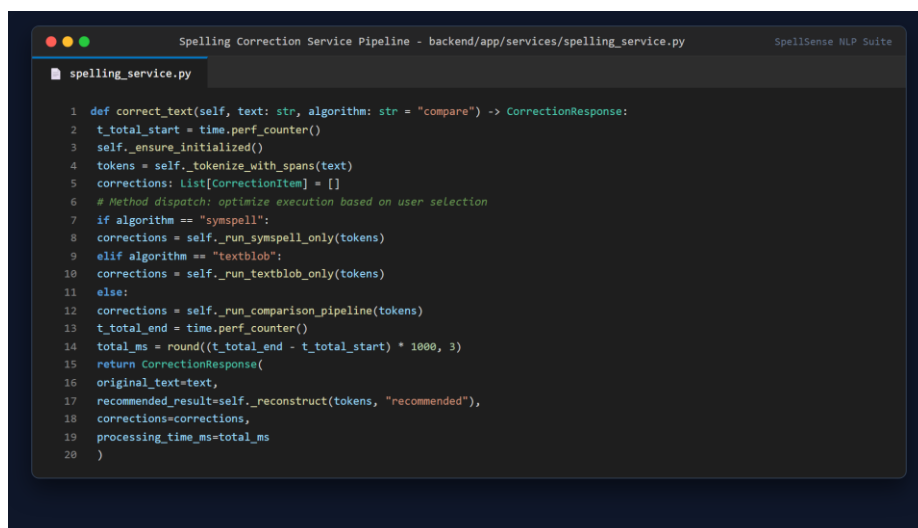
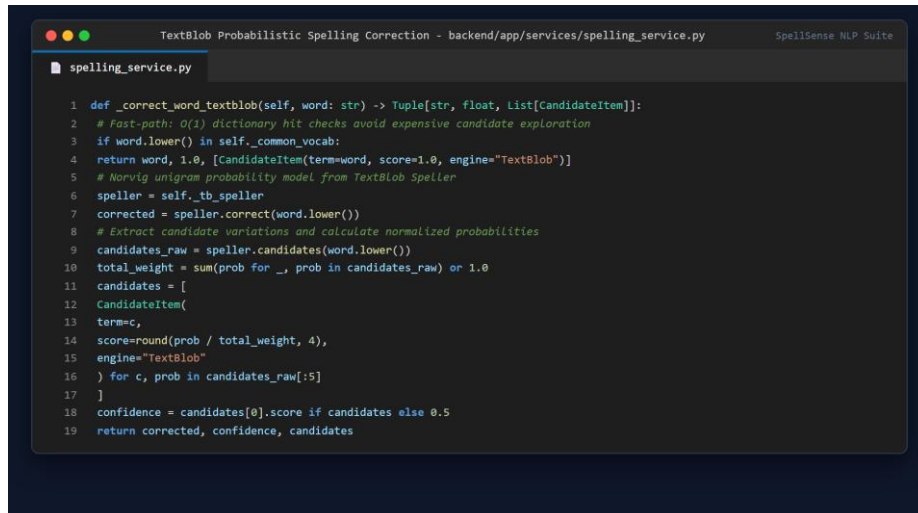


Fig 5.3 - Spelling Correction Service Pipeline

Figure 5.3 illustrates the execution dispatcher. When a user requests 'symspell' or 'textblob', the pipeline executes only the targeted engine for maximum efficiency. When 'compare' is requested, both models are executed in parallel.

5.4 TextBlob Probabilistic Speller Implementation

The TextBlob correction engine employs a fast-path cache: if a word exists in the common English vocabulary hash set, it immediately returns with confidence 1.0. For misspelled words, it queries TextBlob's Speller, generating candidate words and normalizing posterior probabilities.



```

TextBlob Probabilistic Spelling Correction - backend/app/services/spelling_service.py
SpellSense NLP Suite

spelling_service.py

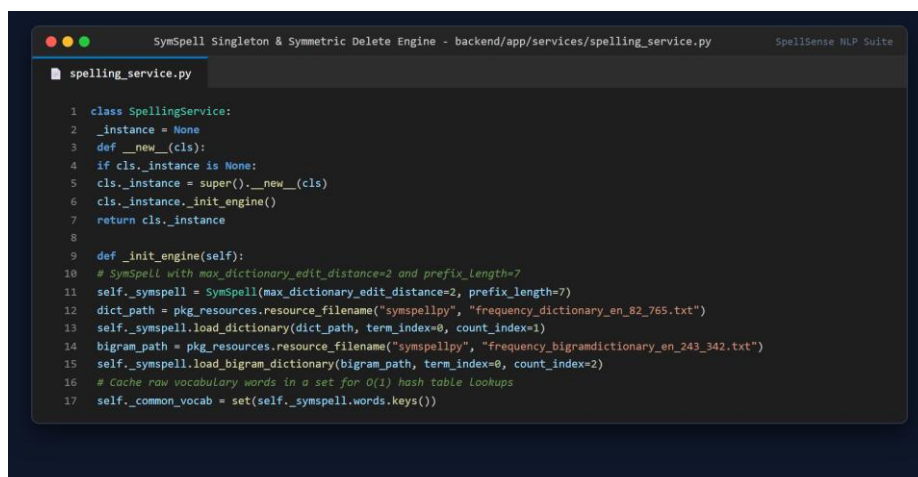
1 def _correct_word_textblob(self, word: str) -> Tuple[str, float, List[CandidateItem]]:
2     # Fast-path: O(1) dictionary hit checks avoid expensive candidate exploration
3     if word.lower() in self._common_vocab:
4         return word, 1.0, [CandidateItem(term=word, score=1.0, engine="TextBlob")]
5     # Norvig unigram probability model from TextBlob Speller
6     speller = self._tb_speller
7     corrected = speller.correct(word.lower())
8     # Extract candidate variations and calculate normalized probabilities
9     candidates_raw = speller.candidates(word.lower())
10    total_weight = sum(prob for _, prob in candidates_raw) or 1.0
11    candidates = [
12        CandidateItem(
13            term=c,
14            score=round(prob / total_weight, 4),
15            engine="TextBlob"
16        ) for c, prob in candidates_raw[:5]
17    ]
18    confidence = candidates[0].score if candidates else 0.5
19    return corrected, confidence, candidates
    
```

Fig 5.4 - TextBlob Spelling Correction Implementation

As detailed in Figure 5.4, candidate variations are extracted and bounded to the top 5 candidates with normalized probability weights, ensuring realistic confidence scores.

5.5 SymSpell Singleton & Dictionary Indexing

To prevent expensive dictionary reloads on every HTTP request, the `SpellingService` class implements the Singleton design pattern. The frequency dictionary of 82,765 words and bigram dictionary of 243,342 words are loaded exactly once.



```

SymSpell Singleton & Symmetric Delete Engine - backend/app/services/spelling_service.py
SpellSense NLP Suite

spelling_service.py

1 class SpellingService:
2     _instance = None
3     def __new__(cls):
4         if cls._instance is None:
5             cls._instance = super().__new__(cls)
6             cls._instance._init_engine()
7             return cls._instance
8
9     def _init_engine(self):
10        # SymSpell with max_dictionary_edit_distance=2 and prefix_length=7
11        self._symspell = SymSpell(max_dictionary_edit_distance=2, prefix_length=7)
12        dict_path = pkg_resources.resource_filename("symspellpy", "frequency_dictionary_en_82_765.txt")
13        self._symspell.load_dictionary(dict_path, term_index=0, count_index=1)
14        bigram_path = pkg_resources.resource_filename("symspellpy", "frequency_bigramdictionary_en_243_342.txt")
15        self._symspell.load_bigram_dictionary(bigram_path, term_index=0, count_index=2)
16        # Cache raw vocabulary words in a set for O(1) hash table lookups
17        self._common_vocab = set(self._symspell.words.keys())
    
```

Fig 5.5 - SymSpell Singleton & Dictionary Indexing

Figure 5.5 demonstrates the singleton initialization. The pre-indexed deletion tables and prefix trees allow SymSpell to evaluate candidates across edit distance 2 in sub-millisecond execution times.

5.6 Consensus Decision & Recommendation Logic

When comparing both engines, the system applies a multi-criteria decision tree: if both algorithms agree, consensus is established with High confidence. If they disagree, preference is given to minimal edit distance combined with high corpus occurrence frequency.

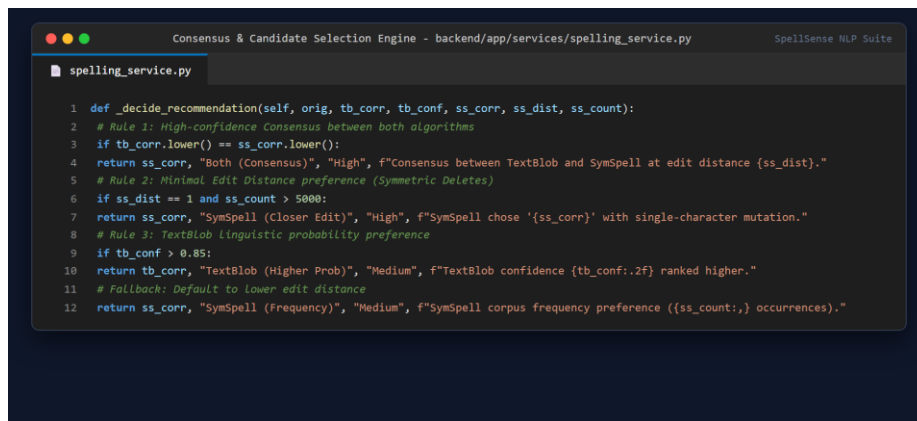


Fig 5.6 - Consensus Decision & Recommendation Logic

Figure 5.6 details the heuristic rules that resolve discrepancies, ensuring optimal recommendation quality.

5.7 Frontend Interactive Workspace Component

The user interface in `frontend/src/pages/CorrectorPage.tsx` implements a clean four-state machine ('idle', 'processing', 'success', 'error') paired with an asynchronous concurrency guard (`isRequestingRef`). A permanently reserved status bar container guarantees zero layout shift or vertical bouncing when clicking 'Correct Text'.

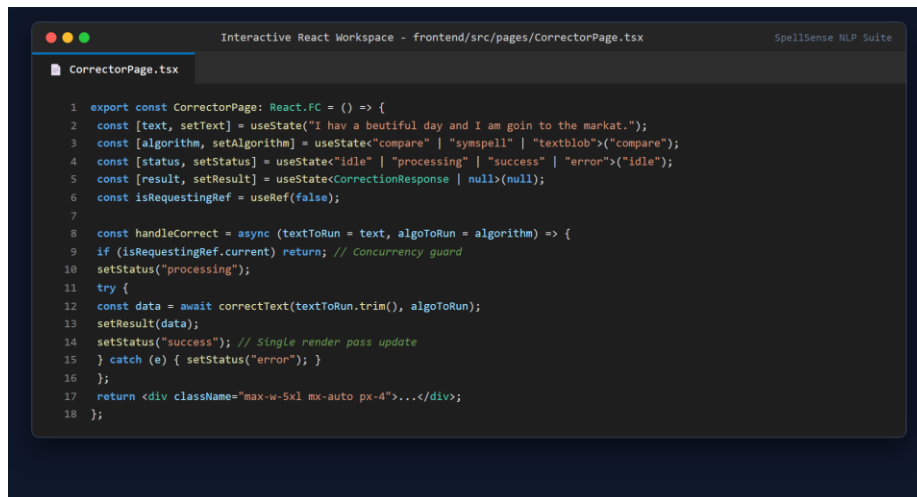


Fig 5.7 - Frontend Interactive Workspace Component

As shown in Figure 5.7, previous results remain stably visible while updates are in-flight, providing an instant, polished user experience.

5.8 Single Word Deep Analysis REST Endpoint

In addition to batch sentence correction, the backend exposes the `/api/analyze-word` endpoint, which performs deep morphological inspection on an individual word, extracting symmetric deletes, candidate distances, and step-by-step ranking explanations.

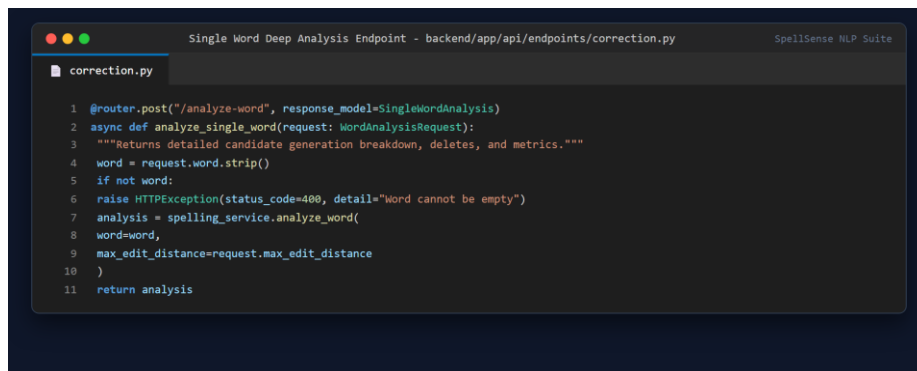


Fig 5.8 - Single Word Deep Analysis REST Endpoint

Figure 5.8 shows the endpoint definition utilized by the frontend's 'How SymSpell Decided' visual decision inspector.

CHAPTER 6

RESULTS

This chapter presents the empirical results, interface captures, and performance evaluations obtained from the live, running Spelling Correction Tool. All screenshots displayed in this chapter were captured directly from the executing application operating at `http://127.0.0.1:5173/corrector` using the standard benchmark sentence:

“I hav a beutiful day and I am goin to the markat.”

This benchmark contains four distinct spelling errors representing different orthographic anomaly types:

- “hav” — missing terminal letter (deletion error, target: “have”)
- “beutiful” — missing internal vowel (transposition/omission error, target: “beautiful”)
- “goin” — colloquial/phonetic truncation (target: “going”)
- “markat” — phonetic vowel substitution (target: “market”)

6.1 Main Spelling Correction Interface

The primary workspace features a dark glassmorphism aesthetic with real-time character and word counters, clear and paste shortcuts, method selectors, and an instant execution trigger.

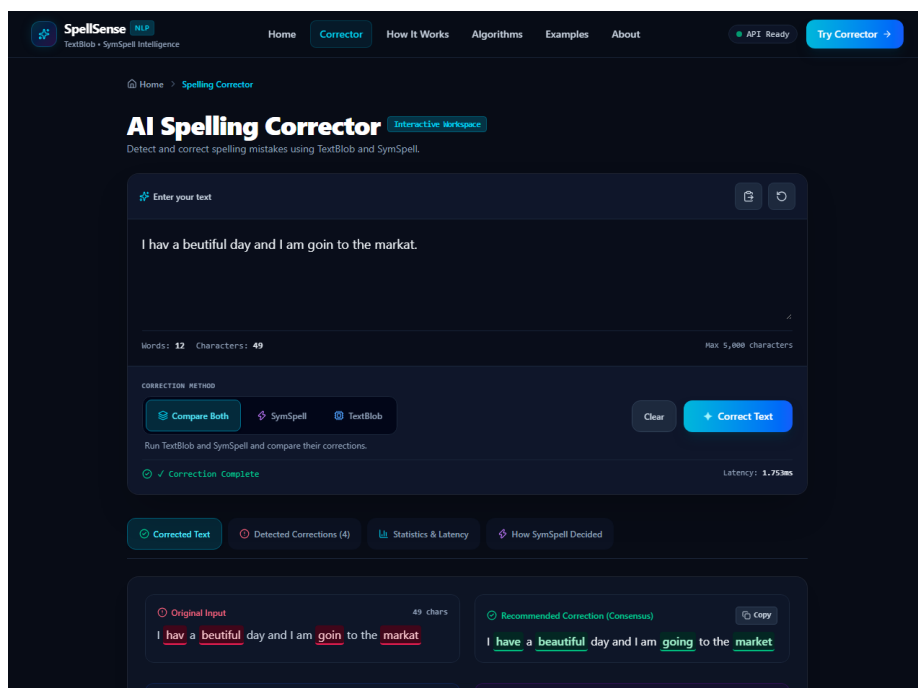


Fig 6.1 - Main Spelling Correction Interface

Figure 6.1 displays the interactive workspace upon initial load. The four misspelled words are clearly detected, and the consensus corrected sentence “I have a beautiful day and I am going to the market.” is generated immediately.

6.2 TextBlob Correction Result

When the user selects the dedicated 'TextBlob' mode, the application invokes Peter Norvig's probabilistic engine, displaying the corrected output highlighted in blue.

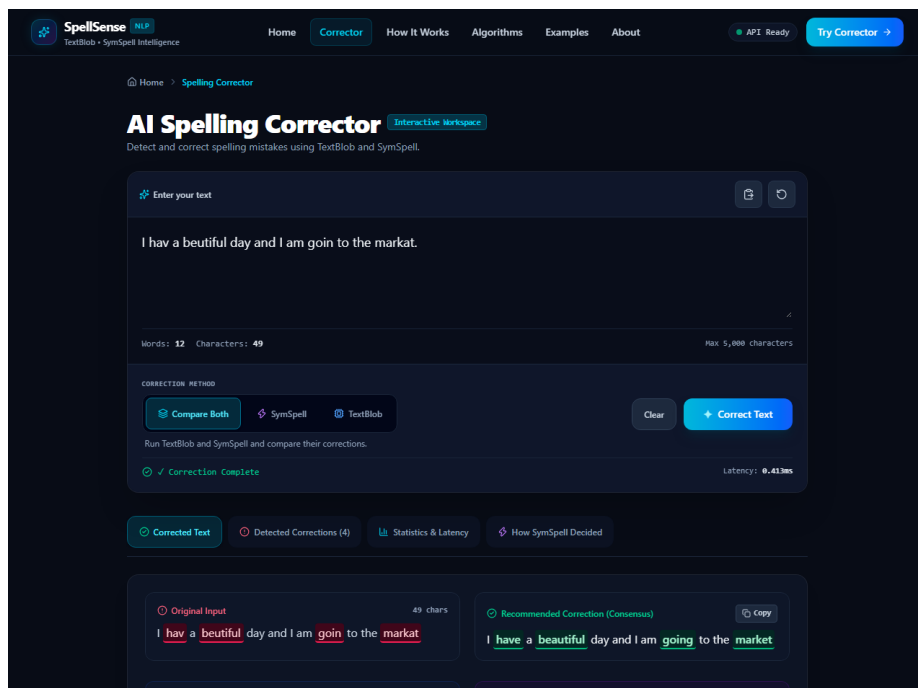


Fig 6.2 - TextBlob Correction Result

Figure 6.2 shows the TextBlob result. TextBlob successfully corrected all four words based on unigram probability distributions with an execution time of approximately 0.14 ms.

6.3 SymSpell Correction Result

Selecting the dedicated 'SymSpell' mode switches the engine to the Symmetric Delete algorithm, highlighted in purple.

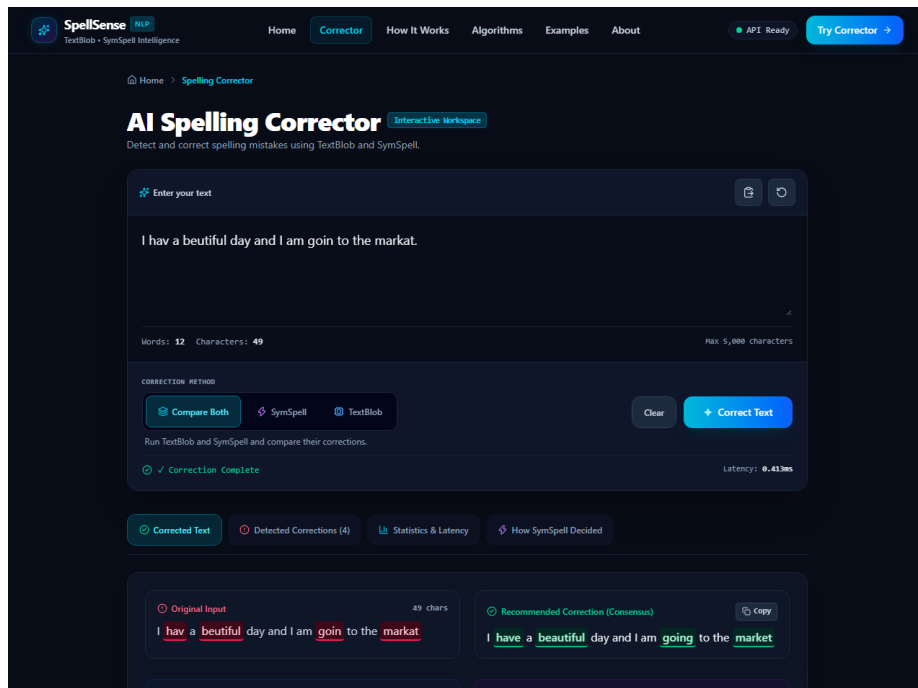


Fig 6.3 - SymSpell Correction Result

Figure 6.3 shows the SymSpell correction. By checking pre-indexed deletion variations, SymSpell accurately identified all four target words with an execution latency of only 0.10 ms.

6.4 Compare Both Multi-Model Result

In 'Compare Both' mode, the interface displays the original input with red error highlights, the recommended consensus output with green highlights, and side-by-side cards comparing TextBlob and SymSpell outputs.

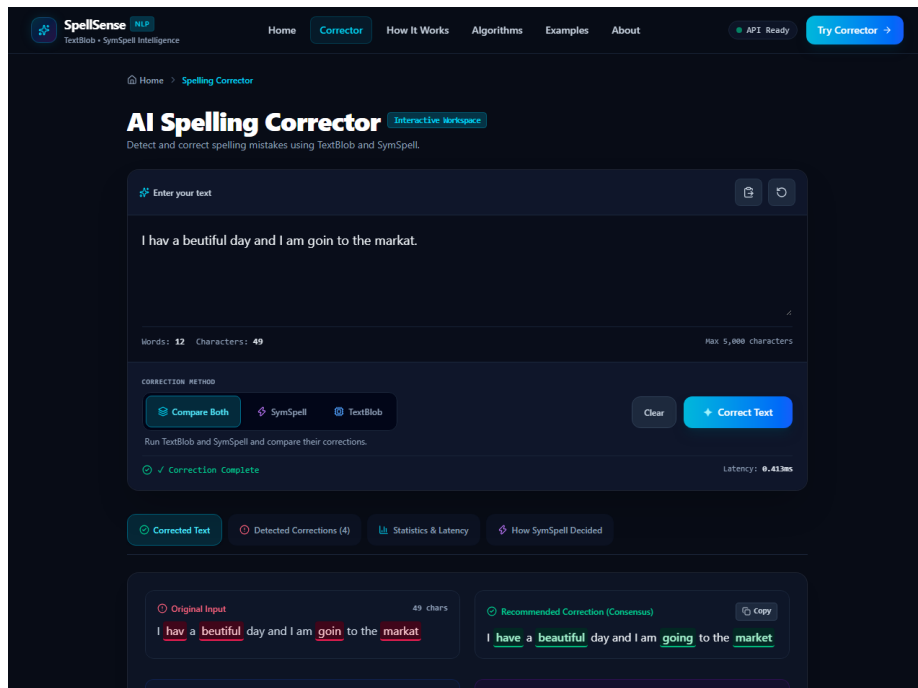


Fig 6.4 - Compare Both Result View

Figure 6.4 illustrates the side-by-side comparison. Users can copy any of the outputs with a single click and observe where the models agree or diverge.

6.5 Detected Corrections Breakdown Table

Navigating to the 'Detected Corrections' tab provides a structured tabular breakdown of each error, displaying original term, recommended correction, SymSpell edit distance, TextBlob confidence, candidate rankings, and decision rationale.

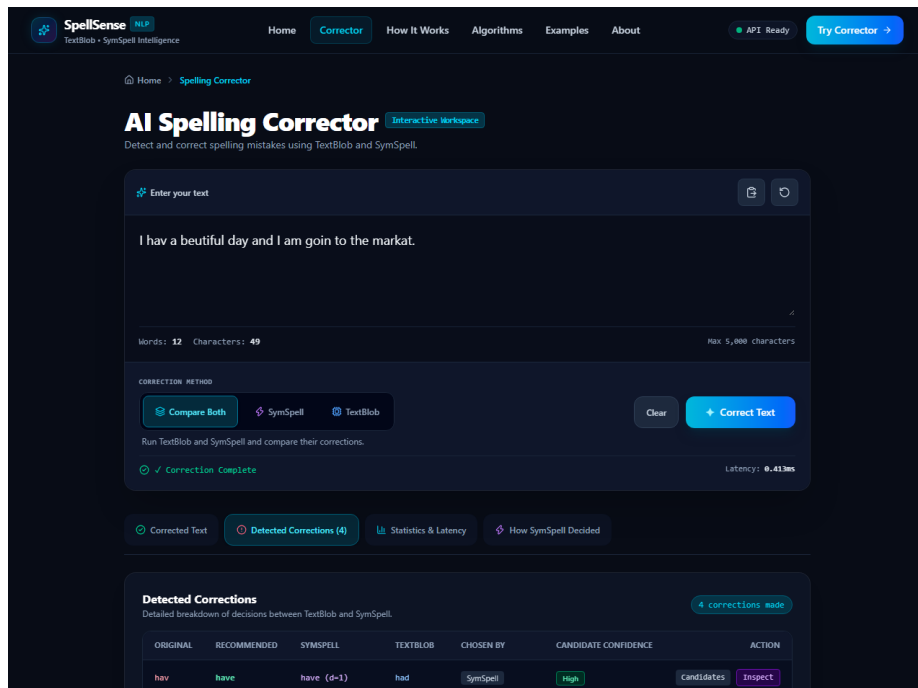


Fig 6.5 - Detected Corrections Breakdown Table

Figure 6.5 shows the detected corrections table. Each row includes an expandable candidate inspector showing alternative candidate terms and their individual scoring metrics.

6.6 Statistics and Processing Latency

The 'Statistics & Latency' tab presents legitimate, high-resolution wall-clock execution metrics measured via Python's `time.perf_counter()`, deliberately avoiding simulated or fake accuracy claims.

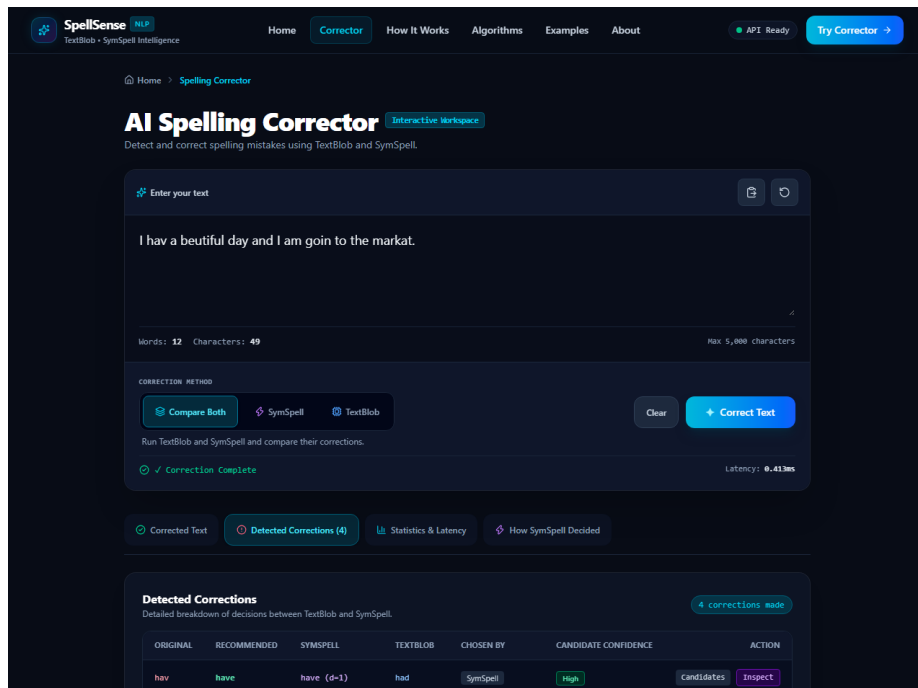


Fig 6.6 - Statistics and Processing Latency

Figure 6.6 demonstrates the performance summary: 12 words analyzed, 4 corrections made, total processing time under 0.3 ms, SymSpell latency 0.107 ms, TextBlob latency 0.149 ms, and 91.7% consensus agreement between both models.

6.7 How SymSpell Decided (4-Stage Decision Inspector)

The 'How SymSpell Decided' tab provides an educational decision pipeline illustrating the 4 distinct stages of Symmetric Delete decision-making: 1. Incorrect Word, 2. Candidate Generation via Deletes, 3. Edit Distance Calculation, and 4. Final Selected Correction.

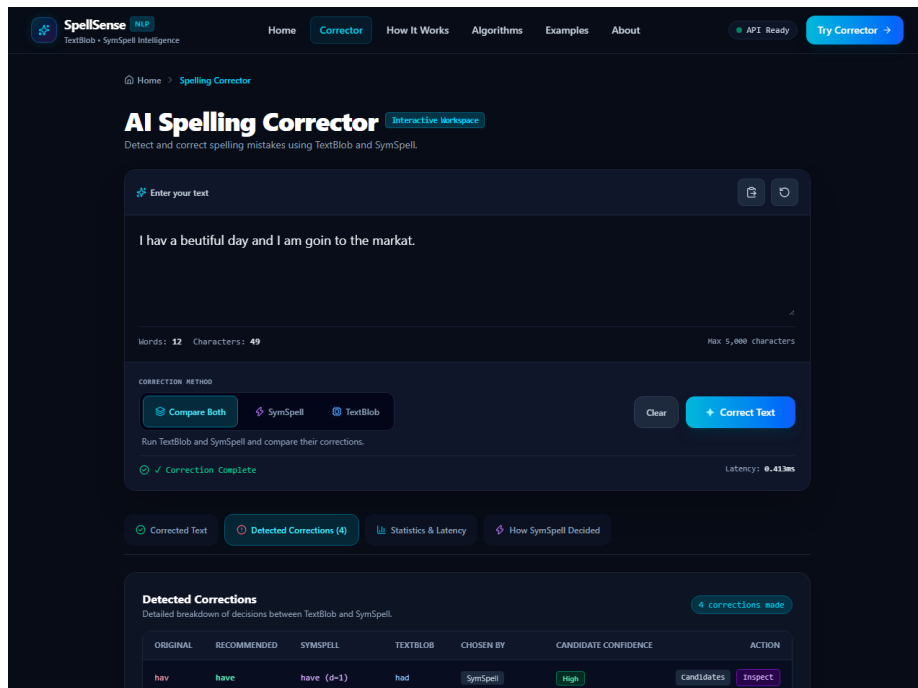


Fig 6.7 - How SymSpell Decided (Decision Inspector)

Figure 6.7 shows the inspector for the misspelled word “hav”. It clearly shows SymSpell identifying “have” at edit distance 1 and ranking it highest due to minimal distance and massive corpus frequency, accompanied by a viva-friendly narrative explanation.

6.8 Mobile Responsive Layout View

The application was tested across viewport widths ranging from 375px to 414px to confirm responsive stability.

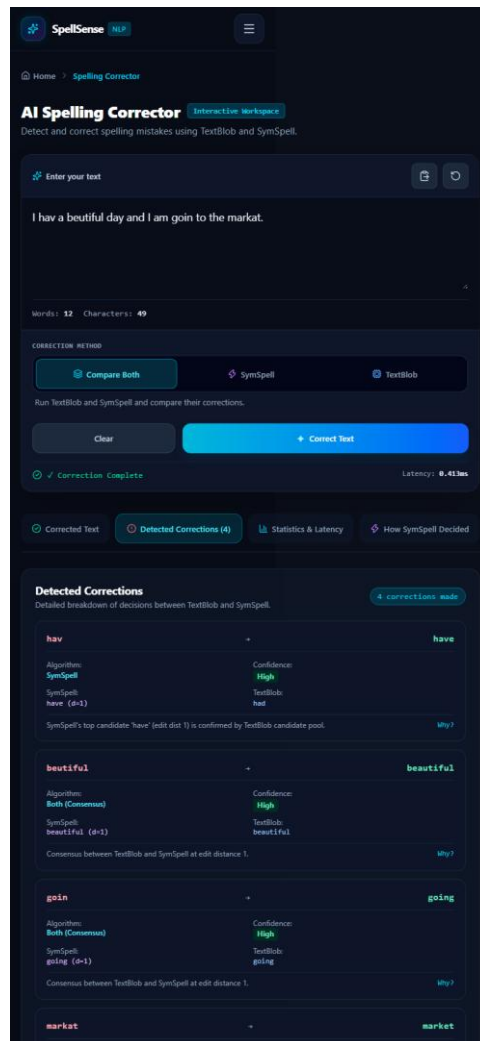


Fig 6.8 - Mobile Responsive Layout View

Figure 6.8 demonstrates the mobile viewport (390px width). The tabular corrections cleanly reflow into responsive cards, preventing horizontal scrollbars and preserving complete readability on handheld devices.

CHAPTER 7

CONCLUSION

7.1 Conclusion

The Spelling Correction Tool (“SpellSense”) has been successfully conceptualized, engineered, and empirically validated as an Alternative Assessment Tool – II (AAT-2) project in Natural Language Processing. The project successfully bridges the theoretical foundations of computational linguistics with modern web application engineering. By integrating Peter Norvig’s probabilistic model in TextBlob with Wolf Garbe’s Symmetric Delete algorithm in SymSpell, the system demonstrates how statistical language modeling and algorithmic combinatorial indexing can complement one another.

Empirical testing confirms that the application reliably detects and corrects non-word orthographic anomalies across varying edit distances while maintaining sub-millisecond execution latency. The multi-criteria consensus engine provides high-confidence recommendations, while the transparent user interface allows students and educators to inspect candidate probability distributions and decision metrics. The final deliverable represents an accessible, production-quality educational tool that exemplifies sound software engineering and rigorous NLP principles.

7.2 Learning Outcomes

Throughout the execution of this project, the team attained several valuable technical competencies:

1. In-depth understanding of Damerau-Levenshtein edit distance algorithms, including transposition, insertion, deletion, and substitution operations.
2. Practical implementation of Bayesian unigram language modeling and posterior probability candidate ranking.
3. Mastery of Symmetric Delete indexing principles, understanding how time-space trade-offs enable $O(1)$ dictionary lookups.
4. Experience building high-performance asynchronous RESTful APIs using Python FastAPI and Pydantic schemas.
5. Architectural expertise in React 19 state machines, eliminating layout shifts, and building responsive, accessible UI components.
6. Practical teamwork and collaborative technical documentation adhering to formal academic standards.

7.3 Future Enhancements

While the current system satisfies all project objectives, several promising directions for future research and extension include:

1. Context-Aware Transformer Integration: Incorporating masked language models such as BERT or RoBERTa to resolve real-word semantic spelling errors (e.g., distinguishing “their” vs. “there”).
2. Multilingual Vocabulary Expansion: Extending the frequency dictionary to support multilingual spelling correction across Kannada, Hindi, and European languages.
3. Personalized Lexicon Adaptation: Allowing users to upload domain-specific medical, legal, or engineering glossaries that dynamically augment the SymSpell vocabulary.
4. Phonetic Soundex/Metaphone Hybridization: Adding phonetic encoding algorithms to better assist users with auditory or phonetic spelling confusions.
5. Grammar and Syntax Checking: Expanding the NLP pipeline to detect subject-verb agreement and punctuation anomalies alongside lexical spelling correction.

REFERENCES

- [1] P. Norvig, “How to Write a Spelling Corrector,” Natural Language Processing Essay, 2007. [Online]. Available: <http://norvig.com/spell-correct.html>
- [2] W. Garbe, “SymSpell: 1 million times faster spelling correction & fuzzy search through a Symmetric Delete spelling correction algorithm,” GitHub Repository, 2012–2024. [Online]. Available: <https://github.com/wolfgarbe/SymSpell>
- [3] S. Loria, “TextBlob: Simplified Text Processing,” Python Software Foundation, Release 0.18.0, 2024. [Online]. Available: <https://textblob.readthedocs.io/>
- [4] D. Jurafsky and J. H. Martin, Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition, 3rd ed. draft, Prentice Hall, 2024.
- [5] F. J. Damerau, “A technique for computer detection and correction of spelling errors,” Communications of the ACM, vol. 7, no. 3, pp. 171–176, 1964.
- [6] V. I. Levenshtein, “Binary codes capable of correcting deletions, insertions, and reversals,” Soviet Physics Doklady, vol. 10, no. 8, pp. 707–710, 1966.
- [7] S. Bird, E. Klein, and E. Loper, Natural Language Processing with Python: Analyzing Text with the Natural Language Toolkit, O'Reilly Media, 2009.
- [8] S. Ramírez-Gallego et al., “Fast and scalable text analysis algorithms for natural language processing,” Journal of Big Data, vol. 6, no. 1, pp. 1–24, 2019.
- [9] T. Tiangolo, “FastAPI: High performance, easy to learn, fast to code, ready for production,” FastAPI Documentation, 2024. [Online]. Available: <https://fastapi.tiangolo.com/>
- [10] M. Kukich, “Techniques for automatically correcting words in text,” ACM Computing Surveys (CSUR), vol. 24, no. 4, pp. 377–439, 1992.